

Syllabus → ① greedy
② DP algorithm
③ Problem Solving using DS & Algo.
④ AVL tree.

Binary Search tree :-

Binary Search tree → A node
 ↓ ↓
left child Right child
(LC) (RC)

$$LC < NODE \leq RC$$

Pseudo Code :-

Procedure Insert (Node n , Node T)

// Node n need to be inserted & T is tree
if (T = Null) return n

while (T ≠ Null) {
 old Temp ← temp
 if (temp.key = n.key) return Error
 if (temp.key > n.key) temp ← temp.left
 else temp ← temp.Right }

key already there

Node {
key
left
right
parent

if (old temp . key < n . key)

old temp . right $\leftarrow$ n

n . parent $\leftarrow$ old temp

n . left $\leftarrow$ Null

n . right $\leftarrow$ Null

else if (old temp . key > n . key) {

old temp . left $\leftarrow$ n

n . parent $\leftarrow$ old temp

n . left $\leftarrow$ Null

n . right $\leftarrow$ Null. }

Time Complexity $\rightarrow$ Maximum travel $\Rightarrow$ height of the tree.

Search Algorithm in BST

Procedure

Search (Node T, key) {

// return either node corresponding to key

// or Null

Do the Code

Tree Traversal-

- ① in Order (r_L, r, r_R)
 - ② pre Order (r, r_L, r_R)
 - ③ post Order (r_L, r_R, r)
-

Procedure

InOrder (Tree T)

```
if (T=NULL) return  
Inorder (T.left)  
  // visit root  
Print (T.key)  
Inorder (T.right)
```

Basically
left subtree is traversed
first

① most left then
its parent & so
on
then right.

↳ similarly for
Pre & Post

Successor & Predecessor (key)

Predecessor. inorder
In traversal, return the key just before the key
Same from Successor.

Return Null if No

Delete Function

Replace the key with Successor
untill it is the last Node
then delete it.

Procedure

→ Delete (Node n) {

// if Node have No child

if (n.right = Null & n.left = Null)

if (n.parent = Null) return Null
else

temp ← n.parent

if (n = temp.left)

// if left child the left temp.left ← Null
// is Null else Right.

else temp.right ← Null

// if Node has only one child.

// if Right child.

if (n.left = Null) {

// Replace the Node with Right child untill

/ Null comes

n.parent ← n.parent

n.right ← n.right

n.right.parent ← n.right

if (n = n.parent.left)

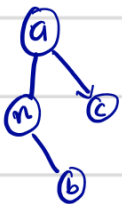
n.parent.left ← n.right

else

n.parent.right ← n.right

// Same code for Right.

(n.right = NULL)



n-parent
key → a
R → c L → n
P → Null

n-right
key → b P → n
R → Null L → Null

n
key → n P → a
L → Null R → b → key → b P → n
R → n L → Null

// have both child
else

// in this case replace node with inorder predecessor or
inorder Successor & delete the corresponding

// replace Successor & Node & w.r.t old position
 $\text{Successor.parent.left} \rightarrow \text{Successor.right}$

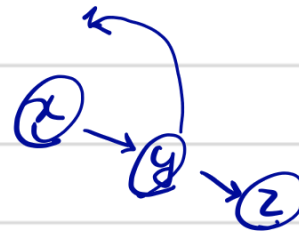
AVL TREE

- ① it is a BST [No duplicate element]
- ② for each Node, Balance factor or $\rightarrow$

$$\frac{|height \text{ of left subtree}| - |height \text{ of right subtree}|}{}$$
is $\{1, 0, -1\}$

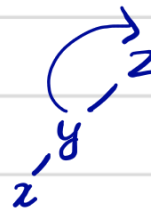
Left Rotation $\rightarrow$

- ① if the element in

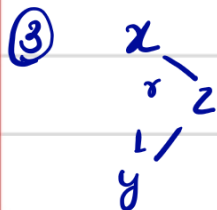


then do
a Left Rot

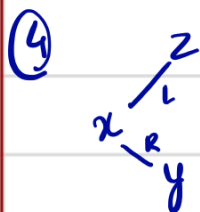
- ② if the element in



then a Right
Rot.



$\rightarrow$ Do a Right Rot then Left.



$\rightarrow$ Left Rot then Right Rotation